Problem 5: Recurrent DeltaNet

GStar Bootcamp

Abstract

Implement DeltaNet by predicting a value, writing its weighted residual, and reading the updated memory.

Starter file: `problem_5.py`. **Checker:** `python autograder.py -p5`. **Reading path:** Overview, P2. Next: P6.

Mathematical background: revise an association instead of adding it

An additive memory can retrieve a value using its key, but similar keys interfere. With unit keys and zero initial state,

$$\mathbf{S}_t^\top \boldsymbol{k}_j = \boldsymbol{v}_j + \sum_{\substack{i \le t \\ i \ne j}} (\boldsymbol{k}_i^\top \boldsymbol{k}_j) \boldsymbol{v}_i.$$

The extra terms show why a matrix memory does not give each token its own cache slot. DeltaNet addresses this by predicting the current association and correcting its error [1].

Let $\widehat{\boldsymbol{v}}_t = \mathbf{S}_{t-1}^\top \boldsymbol{k}_t$ and define a desired interpolated prediction

$$\boldsymbol{v}_t^{\text{target}} = (1 - \beta_t)\widehat{\boldsymbol{v}}_t + \beta_t \boldsymbol{v}_t.$$

Removing the old keyed contribution and writing this target changes memory by

$$\boldsymbol{k}_t (\boldsymbol{v}_t^{\text{target}} - \widehat{\boldsymbol{v}}_t)^\top = \beta_t \boldsymbol{k}_t (\boldsymbol{v}_t - \widehat{\boldsymbol{v}}_t)^\top.$$

For a unit key, the resulting keyed prediction is exactly the target. A read using another key $\boldsymbol{k}_j$ changes by $\beta_t(\boldsymbol{k}_t^\top \boldsymbol{k}_j)(\boldsymbol{v}_t - \widehat{\boldsymbol{v}}_t)$: orthogonal associations are preserved, correlated ones can still change.

There is also an optimization interpretation. For the instantaneous reconstruction loss $\ell_t(\mathbf{S}) = \frac{1}{2}\|\mathbf{S}^\top \boldsymbol{k}_t - \boldsymbol{v}_t\|_2^2$, the gradient is the key outer product with the prediction error. One gradient step with step size β_t gives precisely the delta update. This is adaptation of the *memory state during the forward pass*; implementing model-weight training or backward kernels is not part of this problem.

Before coding. For a repeated unit key, explain why an additive write accumulates old and new values, while a delta write with $\beta_t = 1$ replaces the keyed prediction.

Read a prediction, then correct it

Additive memory writes values without checking what it already predicts. DeltaNet uses a residual [1]:

$$\widehat{\boldsymbol{v}}_t = \mathbf{S}_{t-1}^\top \boldsymbol{k}_t, \tag{5.1}$$

$$\boldsymbol{u}_t = \beta_t(\boldsymbol{v}_t - \widehat{\boldsymbol{v}}_t), \tag{5.2}$$

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \boldsymbol{k}_t \boldsymbol{u}_t^\top, \qquad \boldsymbol{o}_t = \mathbf{S}_t^\top \boldsymbol{q}_t. \tag{5.3}$$

The prediction uses the key; the final output uses the query. These two reads have different purposes and occur on opposite sides of the state update.

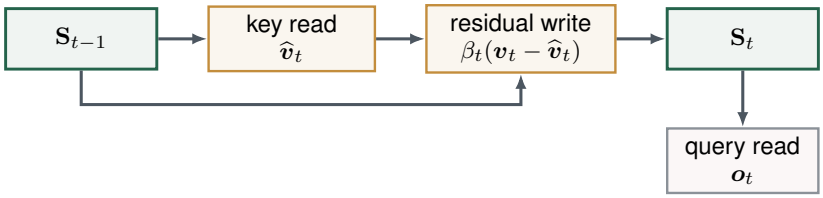


Figure 1. DeltaNet predicts through the key, writes only the weighted prediction error, and then reads the updated state through the query.

Connection to online learning

Consider the reconstruction loss

$$\mathcal{L}_t(\mathbf{S}) = \tfrac{1}{2}\|\mathbf{S}^\top \boldsymbol{k}_t - \boldsymbol{v}_t\|_2^2.$$

Its gradient is $\boldsymbol{k}_t(\mathbf{S}^\top \boldsymbol{k}_t - \boldsymbol{v}_t)^\top$. A gradient step of size β_t yields Eq. (5.3). Equivalently,

$$\mathbf{S}_t = (\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top)\mathbf{S}_{t-1} + \beta_t \boldsymbol{k}_t \boldsymbol{v}_t^\top. \tag{5.4}$$

The transition is identity plus a rank-one modification, a structure we exploit in Problem 6.

Problem 5: a worked correction and student tasks

Repeated normalized keys

Let $\mathbf{k} = (1, 0)^\top$, $D_v = 1$, and suppose $\mathbf{S}_{t-1} = (2, 7)^\top$. For value $v_t = 5$ and $\beta_t = 1$,

$$\widehat{v}_t = 2, \quad u_t = 5 - 2 = 3, \quad \mathbf{S}_t = \begin{bmatrix} 2 \\ 7 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \end{bmatrix} 3 = \begin{bmatrix} 5 \\ 7 \end{bmatrix}.$$

The keyed association becomes 5 while the orthogonal row stays 7. With $\beta_t = 1/2$, the keyed prediction becomes 3.5. More generally, for a normalized key,

$$\mathbf{S}_t^\top \boldsymbol{k}_t = (1 - \beta_t)\widehat{\boldsymbol{v}}_t + \beta_t \boldsymbol{v}_t.$$

Step-by-step implementation

1. **Prediction and residual (30 points).** Reduce the product of the key column and previous state over key channels. Subtract that predicted value tile from the current value tile.
2. **Correction and output (25 points).** Multiply the residual by β_t, write its outer product with the key into the state, then perform the scaled query read.
3. **Explain the correction (20 points).** Using the example above, compare the corrected state with an additive write of value 5. Explain the role of normalized keys.

```
# Reuse the recurrent grid and pointer structure from Problem 2.
for t in range(seq_len):
    # q, k, v, beta are loaded in FP32.
    # TODO P5.A: predict the value using the PREVIOUS state.
    # TODO P5.B: form the value residual and weight it by beta.
    # TODO P5.C: add the corrective outer product to state.
    # TODO P5.D: read the UPDATED state with q and store output.
```

Verification

Use repeated normalized keys to check replacement behavior; use orthogonal basis keys to check independent associations. With $\beta = 0$, the state must remain unchanged and outputs must read that state. Test both zero and nonzero initial states, and compare every output and final state with an independent FP64 recurrence.

Do not swap prediction and output

Predict with the key before writing. Read with the query after writing. Predicting from the updated state changes the residual; reading output before the update omits the current token's contribution.

Problem 5: why normalization makes correction interpretable

The prediction after one correction

For a general key, multiplying the updated memory by that same key gives

$$\mathbf{S}_t^\top \boldsymbol{k}_t = \widehat{\boldsymbol{v}}_t + \beta_t \|\boldsymbol{k}_t\|_2^2 (\boldsymbol{v}_t - \widehat{\boldsymbol{v}}_t). \tag{5.5}$$

If $\|\boldsymbol{k}_t\|_2 = 1$, the new prediction interpolates between the old prediction and the requested value. For $\beta_t = 1$, the keyed prediction is replaced exactly. Without normalization, the effective write strength is $\beta_t \|\boldsymbol{k}_t\|_2^2$, which can exceed one even when $\beta_t \leq 1$.

The memory transition $\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top$ has eigenvalue $1 - \beta_t$ along a unit key and eigenvalue 1 in directions orthogonal to it. The correction targets the keyed association; it does not erase orthogonal associations. Gate forgetting, introduced later, affects those directions as well.

A repeated-key comparison

Let $D_k = D_v = 1$, $k_t = q_t = v_t = 1$ and $S_0 = 0$. Linear attention gives $S_t = t$. DeltaNet with $\beta = 1/2$ gives $S_t = 1 - 2^{-t}$. Gated DeltaNet with the same β and scalar decay $3/4$ gives $S_t = \frac{4}{5}(1 - (3/8)^t)$. The last expression follows from $S_t = (3/8)S_{t-1} + 1/2$.

Do not confuse the key read with the query read

The key read determines what correction to write and has no query scale. The query read determines the emitted output and uses the updated state with the optional `scale` applied in code. Setting $q_t = 0$ must leave the correction unchanged. Setting $\beta_t = 0$ must remove the correction but preserve the query read of existing memory. These are independent controls and should be tested independently.

References

[1] S. Yang et al. *Parallelizing Linear Transformers with the Delta Rule over Sequence Length*. 2024. https://arxiv.org/abs/2406.06484